ReBike: Learning Bike-Share Rebalancing with Masked PPO on Real Citi Bike Demand

Amrit Kumar Yadav (B24CM1008) Raj Kor (B24CM1040)
GitHub: `https://github.com/DeveloperAmrit/rebike`

Abstract

Docking stations that run empty or full turn riders away. We model truck-based rebalancing as a semi-Markov decision process on a simulator calibrated from 561k real Citi Bike trips (Jersey City/Hoboken, Jun–Oct 2024), and train a Proximal Policy Optimization (PPO) agent with invalid-action masking, target-fill actions and a *counterfactual shadow-day reward* that cancels demand noise. On 30 held-out October days the agent loses 7% fewer trips than a grid-tuned threshold policy (36% fewer than no rebalancing) while driving slightly less, and its advantage grows under heavier demand.

1 Problem and Environment

Commuting makes bike flows strongly directional: residential stations empty in the morning while business districts fill up. Operators move bikes with trucks, usually by fixed fill thresholds; optimisation approaches plan overnight or mid-rush routes from forecasts [4]. We ask whether an RL policy can serve more riders per kilometre driven.

Demand model. From the public trip archives [5] we keep the N=60 busiest stations and estimate, for every hour h, Poisson departure rates $\lambda_{h,i}$, origin–destination probabilities $P_{h,ij}$ and median ride times τ_{ij}; dock capacities C_i come from the live GBFS feed. June–September weekdays build the training model, October is held out.

Dynamics. An episode is one weekday (06:00–22:00) simulated in 5-minute ticks. Rentals at an empty station are *lost*; a rider arriving at a full station is redirected to the nearest free dock (a *failed dock*). Each day starts from a *warm start*: one simulated day without a truck and an overnight crew that fixes 30% of each station's imbalance, with the fleet at 40% of docks. A single truck (20 bikes, 18 km/h, 3 min handling) acts at event-driven decision epochs: after an action the clock jumps by the travel time, while demand keeps being simulated.

2 RL Formulation

State $s \in \mathbb{R}^{6N+3}$ (363 dims): fill levels x_i/C_i; one-hot truck position; truck load; sin / cos of time of day; expected net inflow over the next hour ($\sum_j \lambda_{h,j}P_{h,ji} - \lambda_{h,i})/C_i$; bikes in transit arriving within 30 min; truck travel time to every station; and the projected fill in one hour (clipped).

Action $a = (j, k)$, $N{\times}6 = 360$ actions: drive to station j and bring it to target fill $f_k \in \{0.2, 0.35, 0.5, 0.65, 0.8\}$ (moving $\lfloor f_k C_j \rceil - x_j$ bikes, limited by truck load/space), or drive without transferring. Target levels mean the same thing at every station, unlike raw bike counts ($\pm 2, 5, 10$, our first design). Choices that cannot move a bike (empty truck, station already at target, ...) are removed with an action mask [2].

Reward. Policies are scored with the absolute reward $R_t = -\text{lost}_t - 0.2\,\text{km}_t - 0.25\,\text{failed}_t$ (a km and a redirected rider as fractions of a lost trip; demand is exogenous, so −lost ranks policies like +served). For training we use the *counterfactual* reward

$$r_t = R_t - \tilde{R}_t, \quad \tilde{R}_t = -\widetilde{\text{lost}}_t - 0.25\,\widetilde{\text{failed}}_t,$$

where tildes refer to a *shadow day*: the same day, with the same pre-sampled riders, simulated in lock-step without a truck. $\tilde{R}_t$ does not depend on the agent, so the optimal policy is unchanged, but the shared demand noise cancels (validation std. of the return fell from ≈80 to ≈35). Pre-sampling riders also gives *common random numbers*: on a given seed every policy faces identical riders, making all comparisons paired.

Table 1: Held-out October evaluation, mean over 30 days. Lost, km and Full (failed docks) are per day.

Policy	Return	Serv. %	Lost	km	Full
No rebalancing	-413.8	87.3	366	0	190
Threshold (default)	-325.3	90.4	279	125	**87**
Threshold (tuned)	-304.0	91.3	252	136	99
PPO (ours)	**-291.5**	**91.9**	**235**	131	119

3 Algorithm and Baselines

We use Maskable PPO [1, 3]: 8 parallel environments, 256-step rollouts, batch 512, 10 epochs, γ=0.99, λ_{GAE}=0.95, clip 0.2, entropy 0.01, a 256×256 tanh MLP, reward normalisation, learning rate $3{\cdot}10^{-4}$ decayed linearly to 10%, and 2M environment steps (≈55 min on 2 CPU cores). The final agent is the checkpoint with the best return on 30 validation days drawn from the *training* demand model, so the October test month is never used for model selection.

Baselines. *No rebalancing*, and a *threshold* policy: with bikes on board, drive to the station below fill ℓ with the best deficit/distance ratio and fill it to θ; otherwise collect from the station above u. (ℓ, u, θ) is grid-searched on training days (best 0.15/0.75/0.4). All policies are scored on the same 30 seeded October days.

4 Results

PPO achieves the best return (-291.5 vs. -304.0 for the tuned threshold; Table 1) and service rate (91.9% vs. 91.3%; 87.3% without a truck). Paired over the same 30 days it gains $+12.5 \pm 11.6$ return (95% CI) and saves 16.5 ± 11.5 trips per day, winning on 19/30 days, while driving 3% fewer km and moving fewer bikes. Its weak point is full docks (more redirected riders). Fig. 1 shows where the gain comes from: the threshold policy only reacts once a station crosses a threshold and is best at midday, whereas PPO appears to prepare for the evening peak (using the time and inflow features), losing 25–43% fewer trips at 17–18 h.

Ablations (Table 2). Removing the counterfactual reward (v3, same target actions; stopped at 1.25M steps after plateauing) costs ≈44 return and drops PPO below even the default threshold. Bike-amount actions

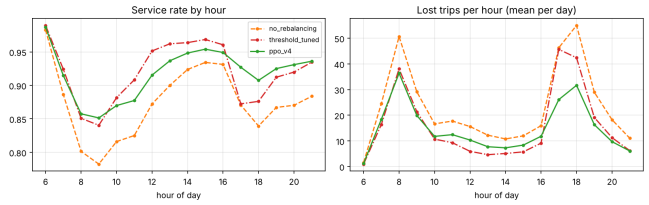


Figure 1: Service rate and lost trips by hour of day (October, 30 days).

Table 2: Ablations on the same 30 October days.

Policy	Return	Serv. %	Lost	km	Full
PPO (ours)	-291.5	91.9	235	131	119
no action mask	-305.7	91.1	257	99	115
absolute reward (v3)	-335.3	89.9	290	69	126
+ amount actions (v2)	-329.7	90.0	288	40	137
+ 4N+3 obs, const. LR (v1)	-328.4	90.3	278	88	131

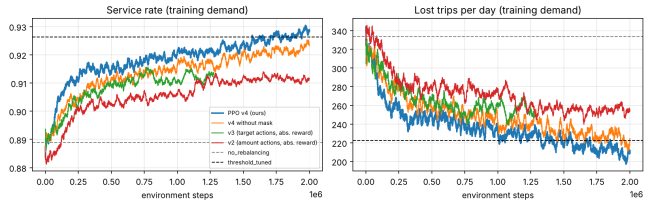


Figure 2: Learning curves (stochastic training policy, rolling mean over 200 days of training demand); dashed: baselines on training demand.

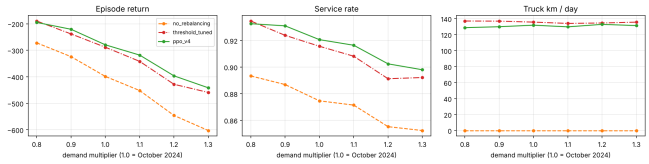


Figure 3: Held-out October days with demand scaled by 0.8–1.3 (15 days each); agents are not retrained.

(v2) and, additionally, the smaller observation without travel times and constant learning rate (v1) stay at the same level, and so do their learning curves (Fig. 2). The counterfactual reward is therefore the decisive ingredient. Training the same agent without the action mask (plain PPO; infeasible transfers are simply clipped by the environment) reaches only the level of the tuned threshold (-305.7), so masking contributes $\approx$14 return.

Robustness (Fig. 3). Scaling October demand by 0.8–1.3 without retraining, PPO's return advantage over the tuned threshold grows from -5.4 at 0.8$\times$ to 9.1 at 1.0$\times$ and 31.9 at 1.2$\times$: the learned policy matters most when the system is under stress.

5 Discussion

A masked PPO agent beats a carefully tuned rule-based policy on unseen days, but only after two design choices: actions expressed as target fill levels and a shadow-day reward that removes exogenous demand noise from the learning signal. The latter is cheap (one extra simulation without a truck) and applies to any RL problem whose reward is dominated by uncontrollable randomness. Limitations: a single truck, 60 stations, a simulator without traffic or weather, and one training seed. The learning curve was still improving at 2M steps, so longer training, several trucks and dock-blocking penalties are natural next steps.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[2] S. Huang, S. Ontañón. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. FLAIRS, 2022 (arXiv:2006.14171).

[3] A. Raffin et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR 22(268), 2021.

[4] E. O'Mahony, D. B. Shmoys. Data Analysis and Optimization for (Citi)Bike Sharing. AAAI, 2015.

[5] Citi Bike System Data. `https://citibikenyc.com/system-data`.